Task 17:

Enable inter-service communication using REST-based communication. Create one microservice that consumes REST APIs exposed by another microservice and handles responses and failures gracefully.

Aim

To enable **inter-service communication** between microservices using **REST APIs**, where one microservice consumes another's API and handles responses and failures gracefully.

Algorithm

1. Identify two services:
    - **Product Service** (Provider)
    - **Order Service** (Consumer)
2. Expose REST API in Product Service.
3. In Order Service:
    - Use RestTemplate or WebClient.
4. Enable service discovery using Eureka.
5. Use service name instead of hardcoded URL.
6. Fetch product details before creating an order.
7. Handle failures using try-catch or fallback logic.
8. Run all services.
9. Test using Postman.
10. Verify successful communication.

Order Service → calls → Product Service → returns product data

Code Implementation

1. Product Service (Provider)

Add API to fetch product by ID

```
@GetMapping("/{id}")
public Product getById(@PathVariable Long id) {
```

```
    return repo.findById(id).orElse(null);
}
```

Order Service (Consumer)

Add RestTemplate Bean

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class Config {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Enable Load Balancing

```
import org.springframework.cloud.client.loadbalancer.LoadBalanced;

@Bean
@LoadBalanced
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

Update OrderController.java

```
package com.orderservice.controller;

import com.orderservice.entity.Order;
import com.orderservice.repository.OrderRepository;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.client.RestTemplate;

@RestController
@RequestMapping("/orders")
public class OrderController {
```

```
    private final OrderRepository repo;
    private final RestTemplate restTemplate;

    public OrderController(OrderRepository repo, RestTemplate restTemplate) {
        this.repo = repo;
        this.restTemplate = restTemplate;
    }

    @PostMapping("/{productId}")
    public Object createOrder(@PathVariable Long productId,
                              @RequestBody Order order) {

        try {
            // Call Product Service
            String url = "http://PRODUCT-SERVICE/products/" + productId;

            Object product = restTemplate.getForObject(url, Object.class);

            if (product == null) {
                return "Product not found!";
            }

            return repo.save(order);

        } catch (Exception e) {
            return "Service unavailable. Please try again later.";
        }
    }
}
```

Output:

Body

Status: 200 OK Time: 929ms Size: 566 B


```
{
  "data": {
    "shop": {
      "id": "gid://shopify/Shop/17681717",
      "name": "johns-apparel",
      "email": "johns@johns-apparel.com"
    }
  },
  "customers": {
    "edges": [
     {
      "node": {
         "id": "gid://shopify/Customer/6581271756",
         "displayName": "Beatrice Alighieri",
         "phone": "+14155550271",
      }
     }
]
```


Response Size	**566 B**
Body	231 B
Headers	335 B
Request Size	**582 B**
Body	220 B
Headers	362 B

All size calculations are approximate

Result

Successfully implemented inter-service communication Used RestTemplate with load balancing Integrated with Eureka for dynamic lookup Handled failures gracefully